



โครงการพัฒนาระบบบริหารจัดการและติดตามสำนวนคดีทุจริต
และประพฤติมิชอบในภาครัฐ

Electronic Case Management Intelligence System (E-CMIS)

รายงานการออกแบบโครงสร้างของระบบงานเพื่อให้รองรับการ
ขยายหรือเพิ่มเติมของระบบได้

(Preliminary Scalability Architecture Design Report)

สัญญาเลขที่ ๑๔/๒๕๖๙

โดย

SS CONSORTIUM

กิจการค้าร่วม SS CONSORTIUM

(บริษัท สามารถคอมเทค จำกัด / บริษัท สมาร์ทเทอร์แวร์ จำกัด)

[illegible]



สารบัญ

บทที่ 1 บทนำโครงการ.....	4
1.1 ความเป็นมา	4
1.2 วัตถุประสงค์ของชุดเอกสาร.....	4
1.3 ขอบเขตระบบงาน	4
บทที่ 2 แนวคิดและหลักการออกแบบโครงสร้างระบบ	5
2.1 แนวคิดการออกแบบเชิงโมดูล (Modular Design).....	5
2.2 แนวคิดสถาปัตยกรรมไมโครเซอร์วิส (Microservices Architecture).....	6
2.3 แนวคิดการออกแบบที่รองรับการเปลี่ยนแปลง (Design for change)	7
บทที่ 3 การออกแบบเพื่อรองรับการขยายตัว	8
3.1 การขยายตัวในระดับแอปพลิเคชัน (Application-Level Scalability).....	8
3.2 การขยายตัวในระดับฐานข้อมูล (Database-Level Scalability)	9
3.3 การขยายตัวในระดับโครงสร้างพื้นฐาน (Infrastructure-Level Scalability)	10
บทที่ 4 การบริหารจัดการค่าคงที่และการตั้งค่าระบบ	11
4.1 สถาปัตยกรรมการจัดการค่าการตั้งค่า	11
4.2 การออกแบบโครงสร้างตารางค่าคงที่.....	12
4.3 ระบบบริหารจัดการพารามิเตอร์ของระบบ	13



บทที่ 1 บทนำโครงการ

1.1 ความเป็นมา

สำนักงาน ป.ป.ท. ดำเนินโครงการพัฒนาระบบ E-CMIS เพื่อสนับสนุนการบริหารจัดการและติดตามสำนวนคดีทุจริตและประพฤติมิชอบให้เป็นระบบ เชื่อมโยงข้อมูลระหว่างกระบวนการ และสามารถติดตามสถานะการดำเนินงานได้อย่างมีประสิทธิภาพ เอกสารเล่มหลักฉบับนี้จัดทำขึ้นเพื่อกำหนดกรอบแนวทางการจัดโครงสร้างของแอปพลิเคชัน (Application Structure) การแบ่งโมดูลการทำงาน และกลไกการตั้งค่าระบบ โดยมุ่งเน้นที่ความยืดหยุ่นในการขยายตัวและลดความยึดติดของโค้ด (Coupling) เป็นสำคัญ

1.2 วัตถุประสงค์ของชุดเอกสาร

- เพื่อกำหนดโครงสร้างการทำงานแบบแยกส่วน (Modular Design): แจกแจงองค์ประกอบของระบบงานหลักให้เป็นโมดูลย่อยที่ทำงานประสานกัน เพื่อให้ง่ายต่อการพัฒนา บำรุงรักษา และลดผลกระทบเมื่อมีการปรับปรุงแก้ไขระบบ
- เพื่อกำหนดโครงสร้างที่รองรับการขยายตัว (Scalability Design): กำหนดกลไกและสถาปัตยกรรมระดับแอปพลิเคชันที่ช่วยให้ระบบสามารถขยาย หรือเพิ่มเติมฟังก์ชันการทำงานใหม่ๆ ในอนาคตได้โดยง่าย ไม่เกิดคอขวดในการประมวลผล
- เพื่อกำหนดแบบกลไกการจัดการค่าคงที่ (Configuration Management): กำหนดแนวทางการบริหารจัดการตัวแปรและค่าคงที่ต่างๆ ของระบบ ให้ผู้ดูแลระบบสามารถปรับเปลี่ยนค่าได้ผ่านเครื่องมือบริหารจัดการ โดยไม่จำเป็นต้องปรับปรุงซอร์สโค้ดใหม่ (No Hardcode)

1.3 ขอบเขตระบบงาน

ขอบเขตของเอกสารฉบับนี้ จะมุ่งเน้นไปที่การอธิบาย "โครงสร้างภายในของระบบงาน (System Components)" และกลไกเชิงเทคนิคที่ช่วยให้ระบบมีความยืดหยุ่น โดยครอบคลุมเนื้อหา ดังนี้

1. โครงสร้างโมดูลแอปพลิเคชัน (Application Modules): การจัดกลุ่มและการแบ่งส่วนการทำงาน (Component Breakdown) ของระบบงานหลักทั้งหมดภายใต้โครงการ E-CMIS
2. โครงสร้างการรองรับการขยายตัว (Scalability Architecture): การออกแบบกลไกเพื่อรองรับภาระงานที่เพิ่มขึ้น ทั้งในระดับการประมวลผล (Compute) และระดับแอปพลิเคชัน (Application)
3. โครงสร้างการจัดการพารามิเตอร์ระบบ (Parameter & Configuration): แนวทางการจัดเก็บและการเรียกใช้ค่าคงที่ของระบบ เช่น ตัวแปรสถานะคดี, รอบเวลาการแจ้งเตือน หรือเงื่อนไขข้อบังคับต่างๆ เพื่อให้ระบบสามารถปรับแต่งพฤติกรรมได้โดยไม่ต้องแก้ไขโปรแกรม



บทที่ 2 แนวคิดและหลักการออกแบบโครงสร้างระบบ

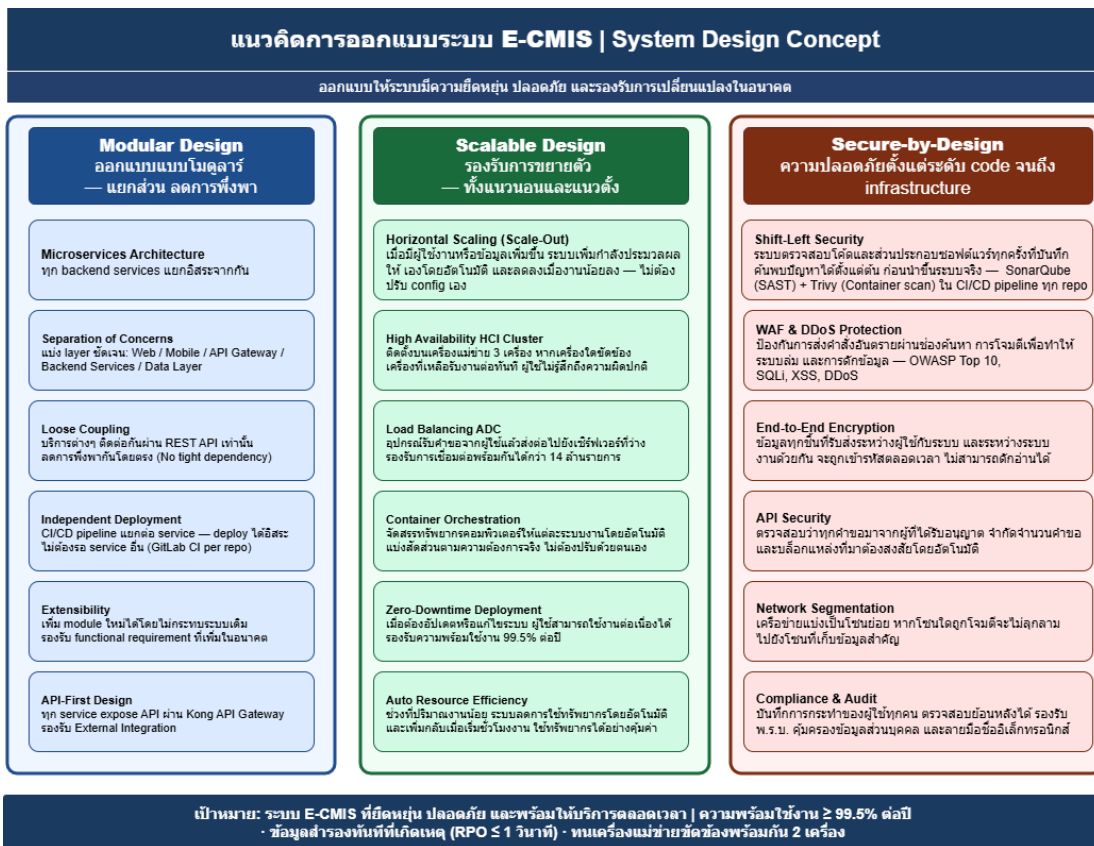
2.1 แนวคิดการออกแบบเชิงโมดูล (Modular Design)

ระบบ E-CMIS ออกแบบโดยยึดหลัก Modular Design โดยแบ่งระบบงานออกเป็นโมดูลย่อยที่ทำงานอิสระต่อกัน (Loosely Coupled) สามารถพัฒนา ทดสอบ และ Deploy แยกกันได้โดยไม่กระทบโมดูลอื่น แต่ละโมดูลสื่อสารกันผ่าน API ที่กำหนดอย่างชัดเจน (Well-defined Interface) ทำให้ทีมพัฒนาสามารถทำงานขนานกันได้อย่างมีประสิทธิภาพ

ในมุมมองของ Business Function ระบบงาน E-CMIS จัดกลุ่มโมดูลออกเป็น 3 กลุ่มหลัก ได้แก่:

- โมดูลจัดการสำนวนคดี (Case Management Modules): รับผิดชอบกระบวนการหลัก ด้านการรับเรื่อง การดำเนินการสืบสวนสอบสวน และการวิเคราะห์ข้อมูลที่เกี่ยวข้องกับสำนวนคดี ซึ่งอาจประกอบด้วยหลายโมดูลย่อยตามกระบวนการของ ป.ป.ท.
- โมดูลบริหารจัดการระบบ (Administration Modules): รับผิดชอบการบริหารผู้ใช้งาน การจัดการสิทธิ์การเข้าถึง การตั้งค่าระบบ และการออกรายงาน เป็นกลุ่มที่ให้บริการด้านการกำกับดูแลระบบแก่ผู้ดูแลระบบโดยเฉพาะ
- โมดูลบริการแกนกลาง (Core Service Modules): รับผิดชอบบริการพื้นฐานที่ทุกโมดูลใช้ร่วมกัน เช่น การยืนยันตัวตน (Authentication) การแจ้งเตือน (Notification) และการบันทึกประวัติการใช้งาน (Audit Logging) โดยบริการเหล่านี้ทำงานอิสระและเรียกใช้ผ่าน API

แนวทางนี้ช่วยให้สามารถเพิ่ม ลด หรือแก้ไขโมดูลได้โดยไม่ต้องหยุดระบบทั้งหมด สอดคล้องกับข้อกำหนด SLA Uptime $\geq 99.5\%$ ต่อปี



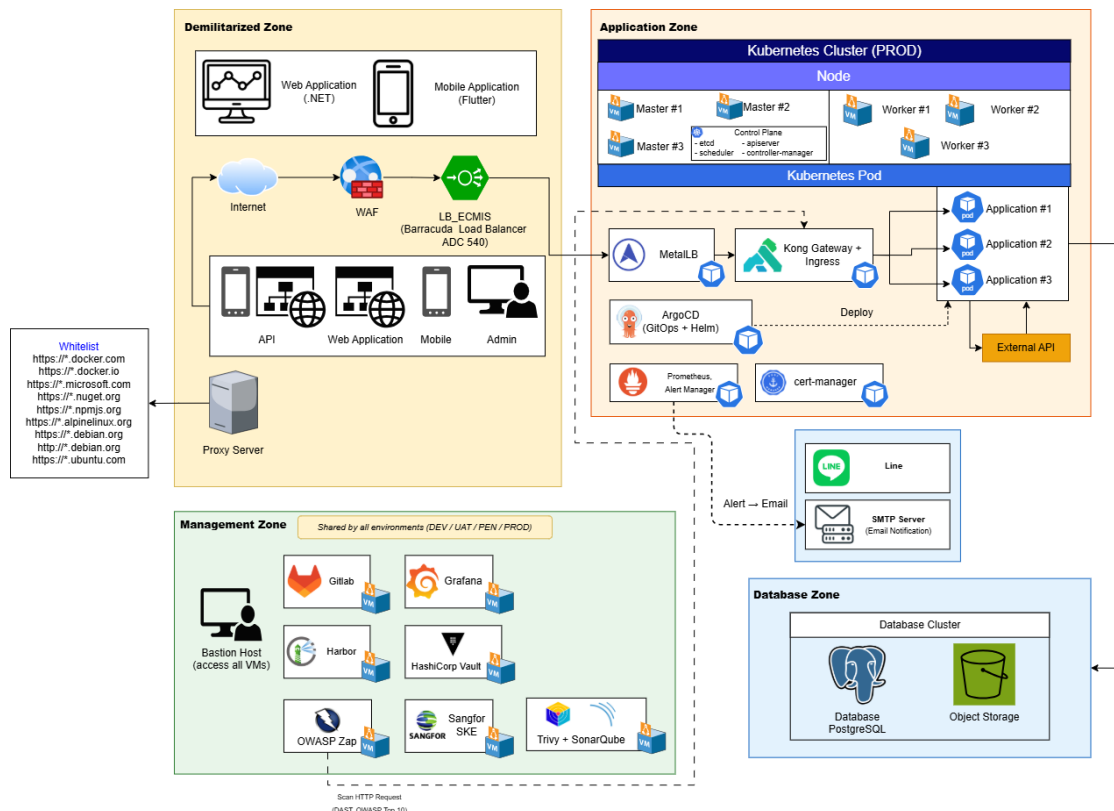
2.2 แนวคิดสถาปัตยกรรมไมโครเซอร์วิส (Microservices Architecture)

ระบบ E-CMIS นำหลักการ Microservices Architecture มาใช้ในการแบ่งระบบงานเป็น Service อิสระที่ Deploy บน Kubernetes แต่ละ Service มีฐานข้อมูลเป็นของตนเอง (Database per Service) เพื่อป้องกันการพึ่งพาข้อมูลข้าม Service และรองรับการ Scale แยกกันได้อย่างเป็นอิสระ Service ใน E-CMIS จัดกลุ่มตาม Functional Domain ออกเป็น 4 กลุ่ม ดังนี้:

กลุ่ม	หน้าที่	ลักษณะการ Scale
Domain Services	รับผิดชอบตรรกะทางธุรกิจของแต่ละกระบวนการงาน เช่น การรับเรื่อง สืบสวน หรือวิเคราะห์ข้อมูล	Scale แยกตาม Load ของแต่ละกระบวนการงาน
Administration Services	บริหารผู้ใช้งาน สิทธิ์ ตั้งค่าระบบ และรายงาน	Scale ต่ำ — ใช้เฉพาะช่วงบริหาร
Core Infrastructure Services	API Gateway, Authentication, Notification, Audit Logging	Scale สูง — ทุก Request ผ่านจุดนี้
Frontend Applications	Web Application และ Mobile Application เป็นช่องทางเข้าถึงระบบ	Scale ตาม Concurrent User

การสื่อสารระหว่าง Service ใช้ REST API ผ่าน Kong API Gateway เป็น Single Entry Point ทำหน้าที่ตรวจสอบ JWT Token, Rate Limiting และ Routing ก่อนส่งคำขอไปยัง Service ปลายทาง (API Gateway, OAuth2)

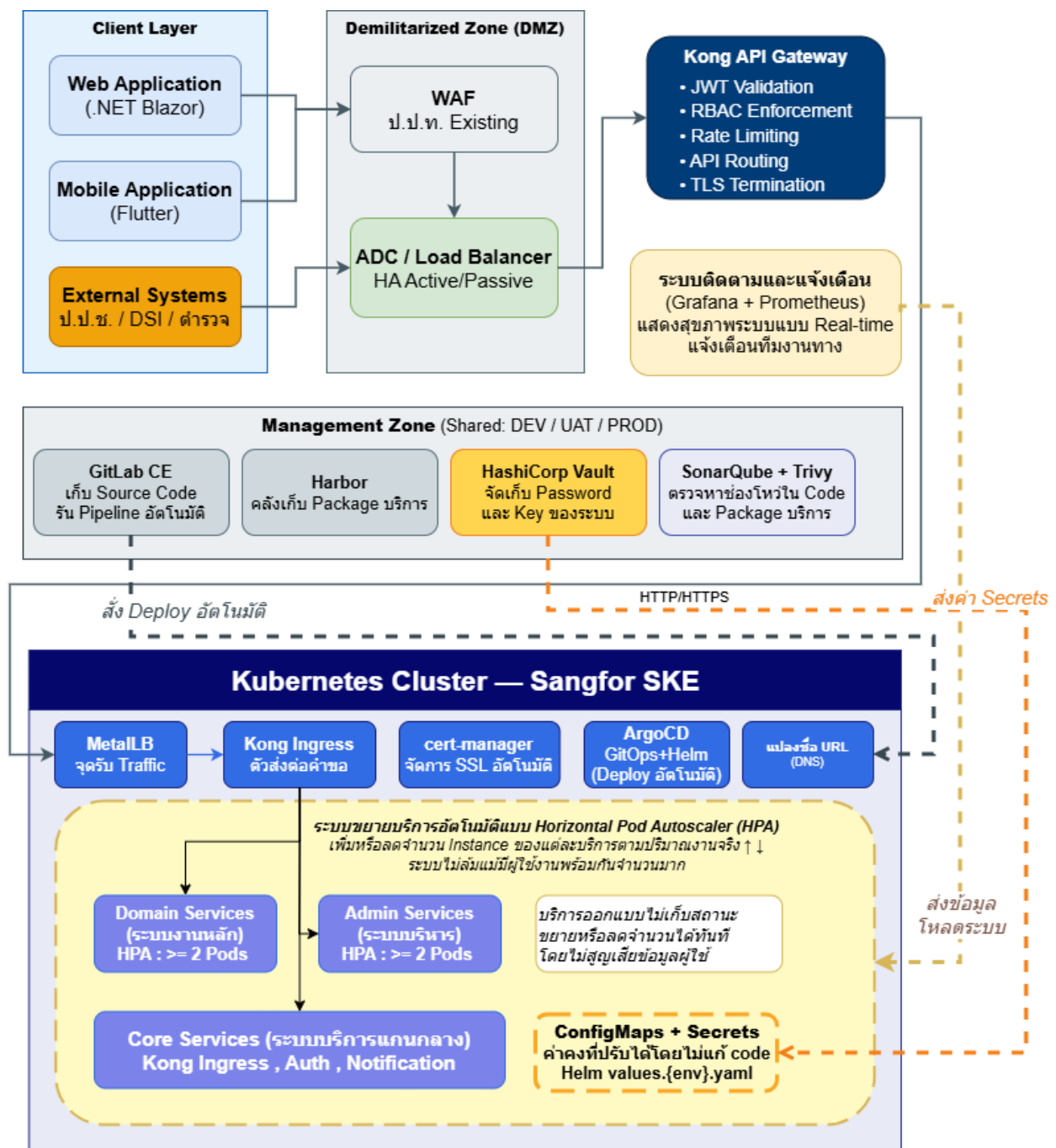
แนวทางนี้ทำให้สามารถเพิ่ม ปรับปรุง หรือแทนที่ Service ใดก็ได้โดยไม่กระทบระบบส่วนอื่น เพราะแต่ละ Service ไม่รู้จักกันโดยตรง — รู้จักกันผ่าน API Contract เท่านั้น



2.3 แนวคิดการออกแบบที่รองรับการเปลี่ยนแปลง (Design for change)

เพื่อให้ระบบ E-CMIS รองรับการพัฒนาตัวและการเปลี่ยนแปลงในระยะยาว การออกแบบยึดหลัก 3 ประการ ดังนี้

- 1) Stateless Services — Service ทุกตัวไม่เก็บ Session State ไว้ใน Memory ทำให้สามารถเพิ่มจำนวน Instance ได้ทันทีโดยไม่เกิดปัญหาข้อมูลไม่สอดคล้องกัน
- 2) Configuration-Driven Behavior — พฤติกรรมของระบบขับเคลื่อนด้วยค่าคงที่ที่ Admin จัดการได้ ไม่ Hardcode ตรรกะทางธุรกิจในโค้ด เช่น รอบเวลาแจ้งเตือน, เงื่อนไขสถานะคดี, ขีดจำกัดการอัปโหลด
- 3) Infrastructure as Code (IaC) — ทุกการ Deploy ผ่าน GitOps (ArgoCD) โดย Manifest ทั้งหมดเก็บใน ecmis-deployment repository ทำให้สามารถ Rollback ไปยังเวอร์ชันก่อนหน้าได้ภายในเวลาไม่กี่นาที



บทที่ 3 การออกแบบเพื่อรองรับการขยายตัว

3.1 การขยายตัวในระดับแอปพลิเคชัน (Application-Level Scalability)

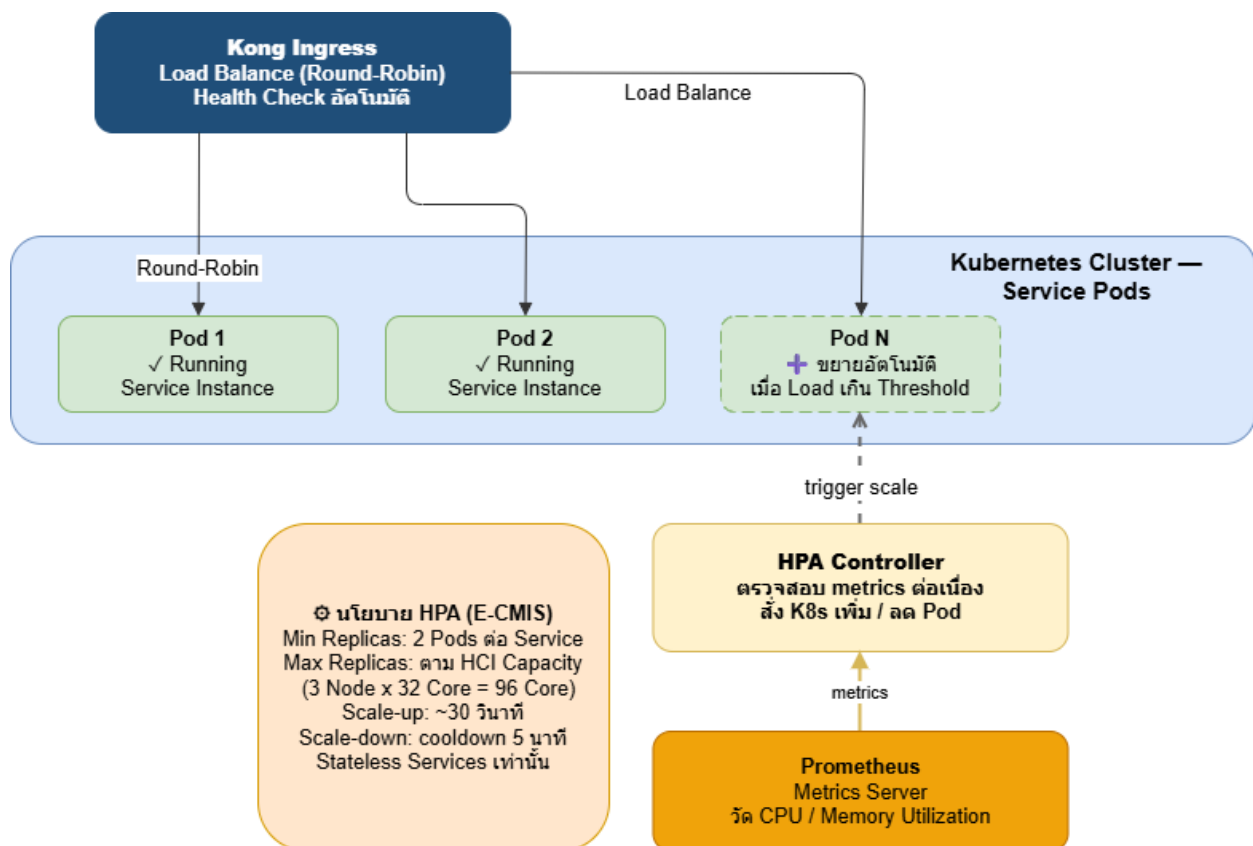
ระดับแอปพลิเคชันใช้กลไก Horizontal Pod Autoscaler (HPA) ของ Kubernetes ปรับจำนวน Pod อัตโนมัติตามภาระงานที่เกิดขึ้นจริง โดยไม่จำเป็นต้องหยุดระบบ

หลักการทำงานของ HPA ในระบบ E-CMIS:

- Metric หลักที่ใช้ตัดสินใจ: CPU Utilization > 70% และ Memory Utilization > 75%
- Minimum Replicas: 3 Pods ต่อ Service (รองรับ HA — หาก Pod หนึ่งล่ม อีก Pod ยังให้บริการได้)
- Maximum Replicas: 10 Pods โดยสมมติจาก CPU Request 0.5 core/pod และ Infrastructure Overhead 25%
- Scale-up ใช้เวลา ~30 วินาที (Kubernetes default) / Scale-down รอ 5 นาทีก่อนลด Pod

Kong API Gateway ทำหน้าที่ Load Balance คำขอระหว่าง Pod ที่ขยายตัว โดยใช้ Round-Robin Algorithm เป็นค่าเริ่มต้น พร้อม Health Check ตรวจสอบ Pod ที่พร้อมให้บริการ

Service ที่คาดว่าจะมีภาระงานสูงสุด เช่น Domain Service ที่รับผิดชอบกระบวนการหลัก ได้รับการตั้งค่า Min Replicas สูงกว่า Service อื่น เพื่อรองรับการทำงานของเจ้าหน้าที่ ป.ป.ท. ในช่วงโม่งทำการ (Peak Hours)



3.2 การขยายตัวในระดับฐานข้อมูล (Database-Level Scalability)

ฐานข้อมูลหลักของระบบ E-CMIS ใช้ PostgreSQL โดยออกแบบให้รองรับการขยายตัวใน 3 ระดับ

ระดับที่ 1 — Connection Pooling ด้วย PgBouncer:

- ป้องกัน Connection Exhaustion เมื่อมี Pod จำนวนมากพร้อมกัน
- ใช้ Transaction Pooling Mode — Connection ถูกคืนกลับ Pool ทันทีที่ Transaction สิ้นสุด
- รองรับ Concurrent Connection ได้มากกว่า Max Connection ของ PostgreSQL เองถึง 5–10 เท่า

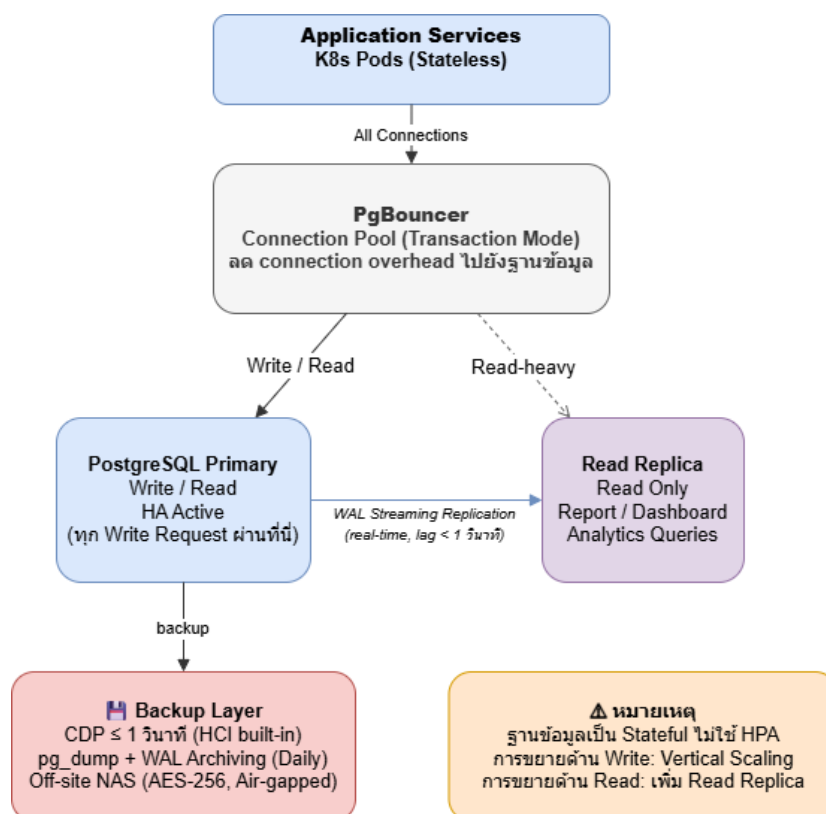
ระดับที่ 2 — Read Replica สำหรับ Queries ที่ต้องการความเร็ว:

- Query ประเภท Report และ Dashboard ส่งไปยัง Read Replica แยกต่างหาก
- ลด Load บน Primary Database ช่วยให้ Write Operations ตอบสนองเร็วขึ้น
- Streaming Replication จาก Primary -> Replica (Lag น้อยกว่า 1 วินาทีในเครือข่าย LAN)

ระดับที่ 3 — Database per Service Pattern:

- แต่ละ Microservice มีฐานข้อมูลแยกเป็นของตนเอง (ไม่แชร์ Schema)
- ป้องกันการขยายตัวของ Service หนึ่งกระทบประสิทธิภาพ Service อื่น
- รองรับการ Migrate Schema อิสระโดยไม่ต้องหยุด Service อื่น

ด้านความมั่นคงของข้อมูล: HCI มีระบบ CDP (Continuous Data Protection) ในตัว รองรับ RPO ≤ 1 วินาที โดยไม่ต้องติดตั้งซอฟต์แวร์เพิ่มเติม



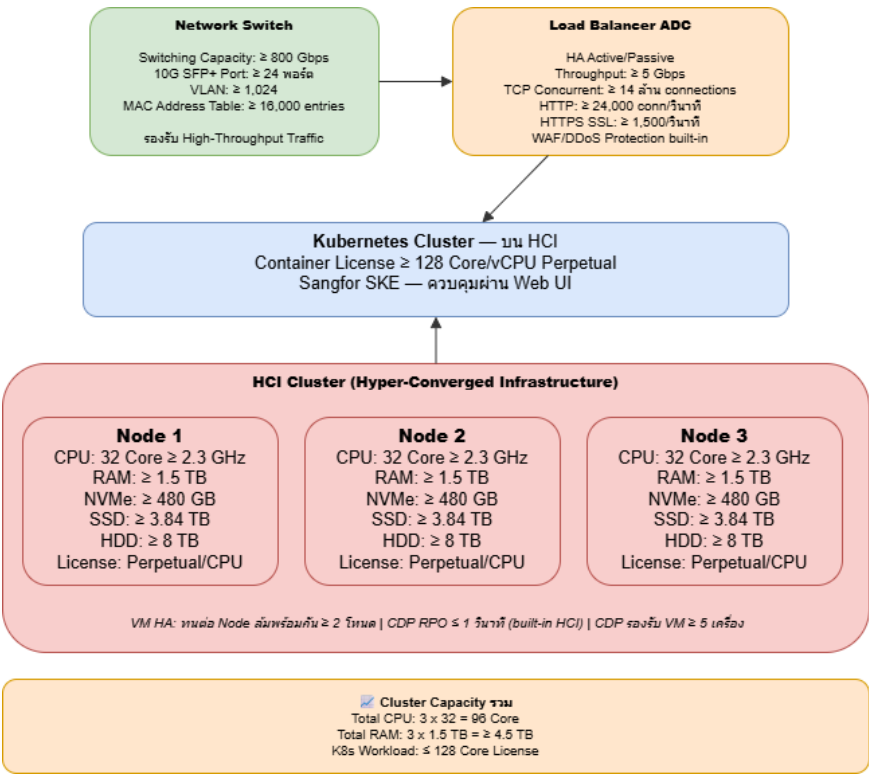


3.3 การขยายตัวในระดับโครงสร้างพื้นฐาน (Infrastructure-Level Scalability)

โครงสร้างพื้นฐานของระบบ E-CMIS ออกแบบให้รองรับการขยายตัวในระดับ Physical และ Virtual เพื่อรองรับการเติบโตของปริมาณงานในอนาคต

อุปกรณ์	รายการ	ข้อกำหนด
HCI Cluster (3 ชุด)	CPU	32 Core ≥ 2.3 GHz ต่อชุด (รวม 96 Core)
	RAM	≥ 1.5 TB ต่อชุด (รวม 4.5 TB)
	NVMe (Tier 1)	≥ 480 GB ต่อชุด
	SSD (Tier 2)	≥ 3.84 TB ต่อชุด
	HDD (Tier 3)	≥ 8 TB ต่อชุด
	VM HA	Node ล้มพร้อมกัน ≥ 2 โหนด ระบบยังทำงานได้ปกติ
	HCI License	Perpetual per Physical CPU ไม่จำกัดจำนวน VM
Load Balancer ADC	Throughput	≥ 5 Gbps
	TCP Concurrent	≥ 14,000,000 รายการ
	HTTP Connections	≥ 24,000 ครั้ง/วินาที
	HTTPS/SSL Offloading	≥ 1,500 ครั้ง/วินาที
	High Availability	Active/Passive พร้อม Failover อัตโนมัติ
Network Switch	Switching Capacity	≥ 800 Gbps
	10G SFP+	≥ 24 พอร์ต
	VLAN	≥ 1,024 รายการ
	MAC Table	≥ 16,000 รายการ

Kubernetes บน HCI: Container License ≥ 128 Core/vCPU Perpetual รองรับการเพิ่ม Worker Node ในอนาคตโดยไม่ต้องหยุด Cluster





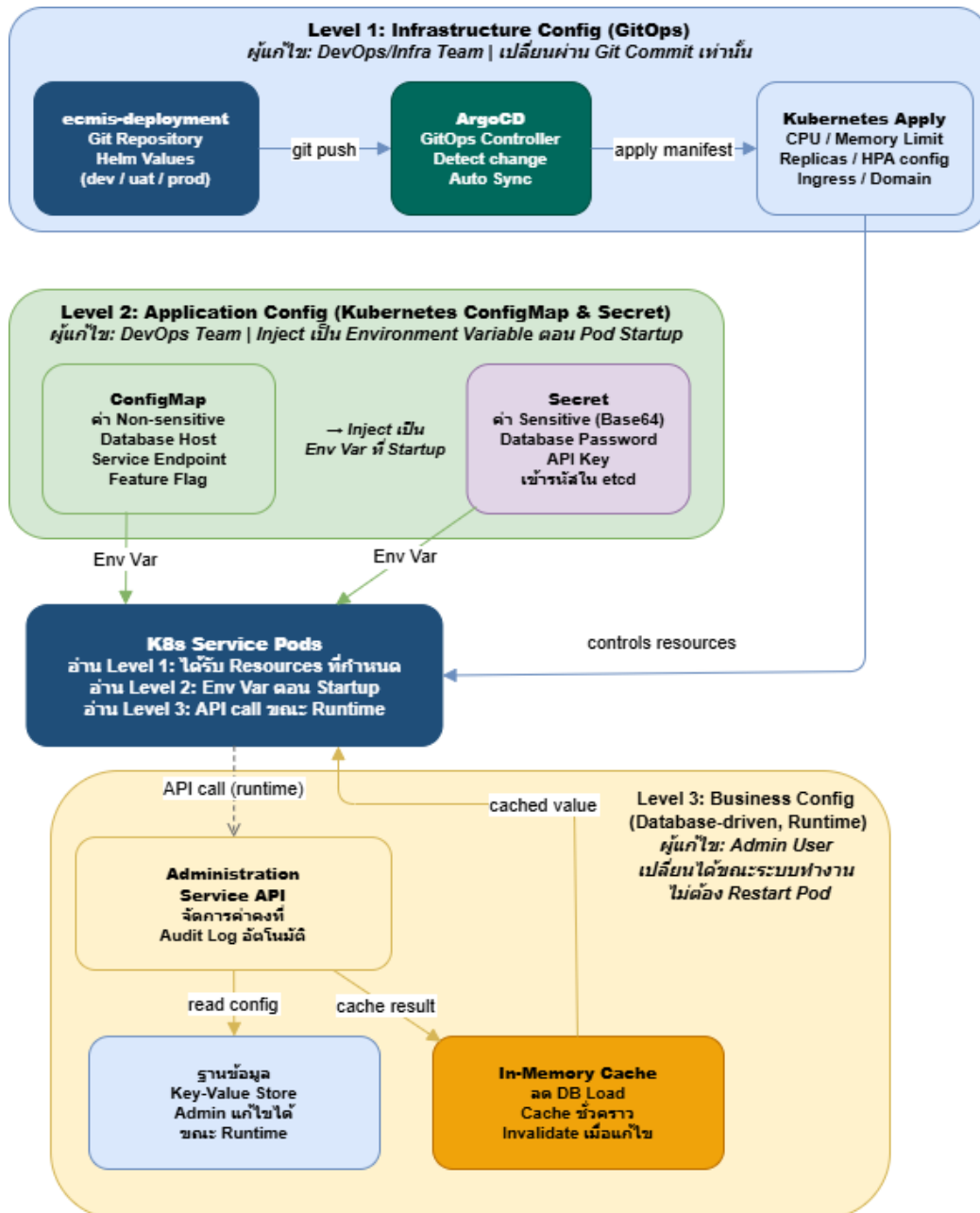
บทที่ 4 การบริหารจัดการค่าคงที่และการตั้งค่าระบบ

4.1 สถาปัตยกรรมการจัดการค่าการตั้งค่า

ระบบ E-CMIS แยกค่าการตั้งค่าออกจาก Source Code อย่างชัดเจน ยึดหลัก 12-Factor App (Config as Environment Variable) เพื่อให้ระบบสามารถ Deploy บน Environment ที่แตกต่างกัน (dev / uat / prod) โดยใช้ Image Docker ชุดเดียวกัน ต่างแค่ค่า Config เท่านั้น

ชั้นการจัดการ Config แบ่งออกเป็น 3 ระดับ:

ระดับ	กลไก / เครื่องมือ	ข้อมูลที่จัดการ	วิธีเปลี่ยนแปลง
Level 1 Infrastructure Config (GitOps)	ecmis-deployment Helm Values (dev / uat / prod) ArgoCD GitOps Controller	CPU/Memory Limit จำนวน Replicas Domain Name Ingress Rule	Git Commit -> ArgoCD Detect -> Auto Sync to Cluster ผู้แก้ไข: DevOps / Infra Team
Level 2 Application Config (K8s ConfigMap & Secret)	ConfigMap (ค่า Non-sensitive) Secret (ค่า Sensitive, Base64 ใน etcd)	Database Host Service Endpoint Feature Flag Database Password API Key	Inject เป็น Env Var ที่ Pod Startup เปลี่ยนแล้ว Redeploy Pod ผู้แก้ไข: DevOps Team
Level 3 Business Config (Database-driven)	ฐานข้อมูล Key-Value Administration Service API In-Memory Cache	เงื่อนไขสถานะคดี รอบการแจ้งเตือน ขนาดไฟล์อัปโหลด กำหนดเวลากระบวนการ	Admin แก้ผ่าน API ขณะ Runtime ไม่ต้อง Restart Pod ผู้แก้ไข: Admin User



4.2 การออกแบบโครงสร้างตารางค่าคงที่

ค่าคงที่ของระบบที่ต้องการความยืดหยุ่นในการปรับเปลี่ยนโดย Admin ออกแบบให้เก็บในฐานข้อมูลในรูปแบบ Key-Value ที่มีโครงสร้างชัดเจน

โครงสร้างข้อมูลค่าคงที่ประกอบด้วย:

- รหัสอ้างอิง (Primary Key) — ระบุค่าคงที่แต่ละรายการได้อย่างไม่ซ้ำกัน
- กลุ่มของค่า (Group) — จัดหมวดหมู่ตามประเภทของกระบวนการงาน เช่น กลุ่มสถานะคดี, กลุ่มการแจ้งเตือน, กลุ่มข้อจำกัดระบบ
- ชื่อค่า (Key) — ระบุค่าคงที่แต่ละรายการภายในกลุ่ม
- ค่า (Value) — ข้อมูลที่ใช้งานจริง ระบบแปลงเป็น Type ที่ถูกต้องตาม Context การใช้งาน

- คำอธิบาย — อธิบายวัตถุประสงค์ของค่านั้นๆ เพื่อให้ผู้ดูแลระบบเข้าใจก่อนแก้ไข
- สถานะ — เปิด/ปิดการใช้งานโดยไม่ลบข้อมูลออกจริง เพื่อรักษาประวัติการตั้งค่า
- ข้อมูล Audit — บันทึกผู้แก้ไขล่าสุด พร้อมวันเวลา รองรับข้อกำหนด PDPA และการตรวจสอบ
- ค่าเดิมก่อนแก้ไข — เก็บ snapshot ค่าก่อนหน้าเพื่อให้สามารถตรวจสอบและ Rollback ได้

ตัวอย่างกลุ่มค่าคงที่ในระบบ E-CMIS:

- กลุ่มสถานะคดี — ควบคุมสถานะเริ่มต้นและการเปลี่ยนผ่านสถานะในกระบวนการดำเนินคดี
- กลุ่มการแจ้งเตือน — กำหนดระยะเวลาและเงื่อนไขที่ใช้ตัดสินการส่งการแจ้งเตือนแก่เจ้าหน้าที่
- กลุ่มข้อกำหนดการอัปโหลด — กำหนดขนาดและประเภทไฟล์ที่อนุญาตในแต่ละส่วนของระบบ
- กลุ่มกำหนดเวลา — ระยะเวลาที่กำหนดสำหรับแต่ละขั้นตอนของกระบวนการงาน

4.3 ระบบบริหารจัดการพารามิเตอร์ของระบบ

Administration Service ทำหน้าที่เป็นศูนย์กลางในการบริหารจัดการค่าคงที่ทั้งหมด โดยมี API และ Admin UI ให้ผู้ดูแลระบบสามารถดูและแก้ไขค่าได้อย่างปลอดภัย

Administration Service รองรับการจัดการค่าคงที่ผ่านชุด API ที่ครอบคลุมความสามารถ:

- ดึงรายการค่าคงที่ทั้งหมด พร้อมความสามารถกรองตามกลุ่มหรือสถานะ
- ดึงค่าคงที่รายการใดรายการหนึ่ง
- แก้ไขค่าคงที่โดยผู้ดูแลระบบที่ได้รับสิทธิ์กำหนด พร้อมบันทึก Audit Log อัตโนมัติ
- ดูประวัติการเปลี่ยนแปลงทั้งหมด เพื่อการตรวจสอบย้อนหลัง (Audit Trail)

กลไก Cache เพื่อประสิทธิภาพ:

- Service อื่นดึงค่า Parameter ผ่าน Administration Service API และ Cache ไว้ใน In-Memory ชั่วคราว
- เมื่อ Admin แก้ไขค่า ระบบส่ง Cache Invalidation Event ให้ Service ที่เกี่ยวข้อง
- รูปแบบนี้ช่วยลด Database Load สำหรับ Parameter ที่ถูกอ่านบ่อยในระหว่างการใช้งานระบบ

การควบคุมการเข้าถึงและ Audit:

- เฉพาะผู้ดูแลระบบที่ได้รับสิทธิ์เท่านั้นที่สามารถแก้ไขค่าคงที่ระบบได้
- ทุกการเปลี่ยนแปลงบันทึก Audit Log โดยอัตโนมัติ (ผู้แก้ไข, ค่าเก่า, ค่าใหม่, เวลา)
- รองรับข้อกำหนด PDPA และ Audit Trail
- ค่าคงที่ที่ไม่ใช้งานแล้วจะถูกปิดการใช้งาน (Soft Delete) แทนการลบจริง เพื่อรักษาประวัติและรองรับการตรวจสอบ